# Labo 09 — Compose : orchestrer la stack

*Théorie — comment remplacer trois `podman run` de dix lignes par un fichier déclaratif, faire démarrer les services dans le bon ordre, et garder le même fichier du poste de développement à la recette.*

## Objectifs

- Comprendre ce que Compose apporte par rapport à des scripts de `podman run`, et ce qu'il ne fait pas.
- Lire et écrire un `compose.yaml` : `services`, `volumes`, `networks`, `secrets`, `depends_on`, `healthcheck`.
- Maîtriser le cycle `up` / `ps` / `logs` / `exec` / `down`, et ce que `down` détruit ou conserve.
- Gérer plusieurs environnements : `compose.override.yaml`, `-f`, `.env`, `podman compose config`.
- Savoir ce que `podman compose` exécute réellement, et ses différences avec Docker Compose.

---

## 1. Le problème : la stack tient en trois commandes… de dix lignes

Au labo 07, la chaîne `web → api → db` demandait un `podman network create`, un volume, trois `podman run` avec leurs `-e`, `-v`, `-p`, `--secret`, dans le bon ordre, puis le nettoyage inverse. Chaque collègue tape ces commandes un peu différemment ; personne ne sait quelle version tourne ; et le jour où il faut ajouter un service, tout le monde relit l'historique du shell.

**Compose** remplace tout cela par un fichier, `compose.yaml`, qui **décrit** l'état voulu : les services, leurs images, leurs variables, leurs volumes, leurs dépendances. Une commande, `podman compose up -d`, crée ce qui manque ; une autre, `podman compose down`, retire tout proprement. Le fichier est versionné avec le code : la stack devient reproductible.

> **→ HISTOIRE**
> En 2014, une petite société, Orchard, publie **Fig**, un outil Python pour décrire plusieurs conteneurs Docker dans un fichier YAML. Docker rachète Orchard la même année ; Fig devient `docker-compose` (2015). En 2020, Docker publie la **Compose Specification**, un standard ouvert, et réécrit l'outil en Go : `docker compose` (v2, sans tiret). `podman-compose`, écrit en Python dès 2019, implémente cette même spécification au-dessus de Podman.

## 2. Anatomie d'un `compose.yaml`

```yaml
name: stack                                  # nom du projet (sinon : le dossier)
services:
  db:
    image: docker.io/library/postgres:16-alpine
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes:
      - pgdata:/var/lib/postgresql/data      # volume nommé (déclaré plus bas)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5
  api:
    image: localhost/api:${API_TAG:-1.0}     # substitué depuis .env
    build:
      context: ../05                         # ou construit depuis ce Dockerfile
    env_file: [api.env]
    depends_on:
      db:
        condition: service_healthy           # attend que db soit "healthy"
  web:
    image: docker.io/library/nginx:alpine
    ports: ["18080:80"]
    volumes:
      - ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro   # bind mount
    depends_on:
      api:
        condition: service_healthy
volumes:
  pgdata:
secrets:
  db_password:
    file: ./db_password.txt
```

Chaque clé sous `services` correspond à une option de `podman run` que vous connaissez déjà : `image` / `build`, `ports` (`-p`), `environment` (`-e`), `env_file` (`--env-file`), `volumes` (`-v`), `secrets` (`--secret`), `restart` (`--restart`), `healthcheck` (`HEALTHCHECK`), `command` / `entrypoint`. Trois choses sont **implicites** : un **réseau** `<projet>_default` est créé et tous les services y sont connectés ; chaque service y est joignable par **son nom** (le DNS du labo 07) ; et les conteneurs s'appellent `<projet>_<service>_1`.

**→ YAML**
Le YAML repose sur l'**indentation** (deux espaces, jamais de tabulation) : `db:` sous `services:` est un service, `pgdata:` sous `volumes:` est un volume. Une valeur sans rien après les deux-points (`pgdata:`) est vide : « crée-le avec les options par défaut ». Les listes s'écrivent avec `-` ou entre crochets ; les chaînes qui contiennent `:` (comme `"18080:80"`) se mettent entre guillemets, sinon YAML y voit une clé. `podman compose config` affiche le fichier tel que l'outil l'a compris : c'est le premier réflexe quand « ça ne fait pas ce que j'ai écrit ».

## 3. Le cycle de vie

| Commande | Effet | Note |
|---|---|---|
| `podman compose up -d` | crée réseau, volumes, secrets, conteneurs ; démarre dans l'ordre des `depends_on` | relancée, ne recrée que ce qui a changé |
| `podman compose ps` | état des services du projet | `podman ps` montre les mêmes conteneurs |
| `podman compose logs -f api` | logs d'un ou de tous les services | préfixés par `[api] \|` |
| `podman compose exec db psql …` | commande dans un service | `podman exec stack_db_1 …` fait pareil |
| `podman compose stop` / `start` / `restart` | sans détruire | comme `podman stop/start` |
| `podman compose down` | arrête et supprime conteneurs, réseau, pod | **garde les volumes** |
| `podman compose down -v` | idem, **plus les volumes** | la base disparaît |
| `podman compose build` | construit les services qui ont `build:` | `up --build` enchaîne |
| `podman compose config` | affiche la configuration fusionnée et substituée | à lire avant `up` |

Le **nom du projet** est la clé de tout : il préfixe conteneurs, réseau et volumes. Par défaut c'est le nom du dossier ; `name:` dans le fichier (ou `-p`) le fixe. Lancer le même fichier depuis un dossier renommé crée **un second projet**, avec un volume `autre_pgdata` vide : « la base a disparu » est le symptôme classique.

## 4. Démarrer dans le bon ordre

`depends_on: [db]` seul garantit que `db` est **démarré** avant `api` — pas qu'il est prêt. PostgreSQL met deux secondes à accepter des connexions ; Spring Boot, lui, tente de se connecter immédiatement et s'arrête sur `Connection refused`. D'où la forme longue : `condition: service_healthy`, qui attend que le **healthcheck** de `db` réussisse.

Un healthcheck est une commande exécutée **dans** le conteneur à intervalle régulier ; son code de sortie décide : `0` sain, autre chose malade. Après `retries` échecs consécutifs, l'état passe à `unhealthy`. Il peut venir du Dockerfile (`HEALTHCHECK`, labo 10) ou du `compose.yaml`, qui a priorité. La commande doit **exister dans l'image** : `curl` n'est pas dans `eclipse-temurin:21-jre-alpine`, `wget` (BusyBox) si.

**→ SPRING BOOT**
Le démarrage ordonné n'exonère pas l'application : la base peut tomber **après**. Un service robuste réessaie ( `spring.datasource.hikari.initializationFailTimeout=-1` démarre sans base et attend), et `/actuator/health` devient le healthcheck naturel de l'API — c'est lui que `web` attend dans notre stack. En Kubernetes (labo 12), ces mêmes sondes s'appellent *readiness* et *liveness probes*.

**PODMAN**
En rootless, Podman exécute les healthchecks avec des **timers systemd** : sans session systemd (une distribution WSL sans `systemd=true` ), l'état reste `starting` pour toujours, et `condition: service_healthy` bloque `up` . Vérifiez `systemctl --user status` avant d'accuser Compose. Docker, lui, planifie les healthchecks dans son daemon.

## 5. Un fichier, plusieurs environnements

Trois mécanismes se combinent :

- **`compose.override.yaml`** est fusionné **automatiquement** avec `compose.yaml` : on y met ce qui n'est vrai qu'en développement (publier la base sur `127.0.0.1:5432` , monter les sources, un profil `dev` ).
- **`-f compose.yaml -f compose.prod.yaml`** fusionne explicitement les fichiers donnés, dans l'ordre, et **ignore alors l'override** : `restart: unless-stopped` , pas de port pour la base, profil `prod` .
- **`.env`** à côté du fichier fournit les variables **substituées dans le YAML** : `${API_TAG:-1.0}` vaut `API_TAG` si défini, sinon `1.0` . `API_TAG=2.0 podman compose config` le montre.

**PIÈGE**
`.env` et `env_file` n'ont **rien à voir**. `.env` est lu par Compose pour remplacer les `${…}` du fichier YAML ; il n'entre pas dans les conteneurs. `env_file: [api.env]` est passé au conteneur comme `--env-file` . Un `APP_MESSAGE=…` placé dans `.env` n'arrive jamais à l'API. Deuxième piège : `podman compose exec` relit les fichiers par défaut pour injecter l'environnement du service — si le projet a été lancé avec `-f` , donnez les **mêmes** `-f` à `exec` (ou utilisez `podman exec` ), sinon vous verrez les variables de l'override.

## 6. Ce que `podman compose` exécute

**PODMAN**
`podman compose` n'est qu'un **relais** : il cherche `podman-compose` (Python, `apt install podman-compose` ; prenez une version ≥ 1.1, par `pipx` si celle d' `apt` est ancienne) ou `docker-compose` , et lui passe la main — d'où la bannière `>>>> Executing external compose provider` . Différences visibles avec Docker Compose : les conteneurs se nomment `stack_api_1` (Docker : `stack-api-1` ) ; podman-compose regroupe les conteneurs dans un **pod** `pod_stack` sans conteneur infra (un simple regroupement, chaque service garde son IP) ; `down` ne sait pas retirer les instances créées par `--scale` ; et `restart:` est appliqué par le conteneur, sans daemon pour le relancer après `wsl --shutdown` (labo 03). Pour un vrai démarrage automatique, Quadlet (labo 10). L'autre voie, plus fidèle à Docker : `systemctl --user enable --now podman.socket` et le vrai `docker compose` v2 pointé sur `DOCKER_HOST=unix://$XDG_RUNTIME_DIR/podman/podman.sock` .

## 7. En entreprise

- **Compose est l'outil du poste et de la CI** : un `podman compose up -d` donne à chaque développeur la stack complète en une minute, et la CI l'utilise pour les tests d'intégration ( `up`

`-d`, tests, `down -v`).

- **Il convient aux petits déploiements** : une VM, une application, `compose.prod.yaml` et `restart: unless-stopped`. Il ne fait ni haute disponibilité, ni mise à jour sans interruption, ni répartition sur plusieurs machines : c'est la frontière avec Kubernetes (labo 11).
- **Tout ce que vous avez écrit ici se transpose** : un service devient un Deployment, un volume nommé un PersistentVolumeClaim, un healthcheck une *probe*, `depends_on` une *readiness*, `ports` un Service. Compose est le brouillon de vos manifestes.
- **Les secrets** restent hors du fichier : `secrets:` pointe vers un fichier ignoré par Git, ou vers un secret du moteur (`external: true`).

---

## À retenir

- Compose décrit l'état voulu dans un fichier versionné ; `up` crée ce qui manque, `down` retire tout sauf les volumes.
- Un service = un `podman run` traduit en YAML ; le réseau et le DNS sont implicites, par nom de service.
- Le nom du projet préfixe tout ; `name:` évite les projets fantômes et les volumes « disparus ».
- `depends_on` seul attend le démarrage ; `condition: service_healthy` attend un healthcheck qui réussit — avec une commande présente dans l'image.
- `compose.override.yaml` est automatique, `-f` est explicite et l'ignore ; `podman compose config` montre le résultat.
- `.env` substitue dans le YAML, `env_file` entre dans le conteneur : deux choses différentes.
- `podman compose` relaie vers podman-compose : noms en `_`, pod de regroupement, pas de `scale` à la baisse, `restart` sans daemon.

## Vocabulaire

**Compose** : outil et format pour décrire une application multi-conteneurs. — **projet** : ensemble nommé de ressources créées depuis un fichier Compose. — **service** : un conteneur (ou plusieurs instances) décrit dans `services:`. — **`depends_on`** : ordre de démarrage, avec ou sans condition. — **healthcheck** : commande périodique qui décide de l'état `healthy` / `unhealthy`. — **override** : fichier fusionné par-dessus `compose.yaml`. — **substitution** : remplacement des `${VAR}` depuis `.env` ou le shell. — **`podman compose config`** : affichage de la configuration fusionnée. — **podman-compose** : implémentation Python de la Compose Specification pour Podman. — **pod de projet** : regroupement `pod_<projet>` créé par podman-compose.